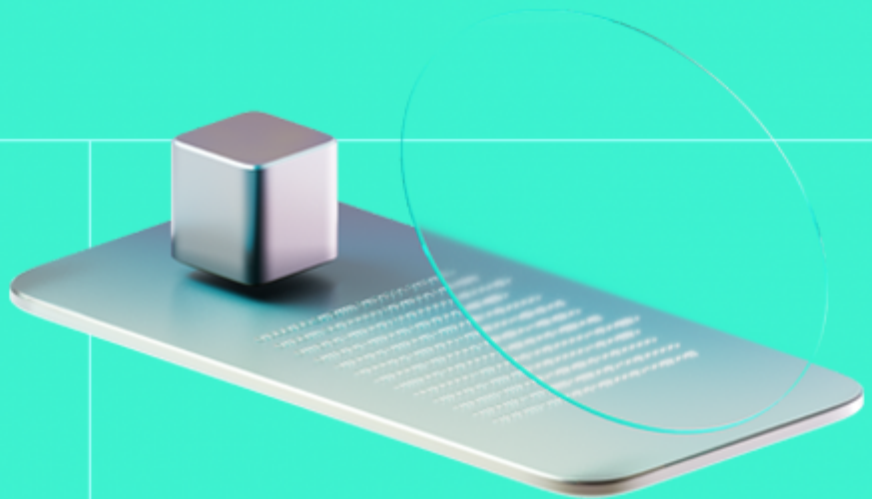




Smart Contract Code Review And Security Analysis Report

Customer: RYT

Date: 23/12/2025



We express our gratitude to the RYT team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

RYT is the layer 1 blockchain platform built for institutional adoption and mass inclusion, powered by Proof of Majority.

Document

Name	Smart Contract Code Review and Security Analysis Report for RYT
Audited By	Kornel Świątłowski, Khrystyna Tkachuk
Approved By	Ataberk Yavuzer
Website	https://ryt.io/
Changelog	12/12/2025 - Preliminary Report
	23/12/2025 - Final Report
Platform	Ethereum
Language	Solidity
Tags	ERC721, Signature, Decentralized Identity
Methodology	https://docs.hacken.io/methodologies/smart-contracts

Review Scope

Repository	https://github.com/ryt-io/DID-Contract-
Initial Commit	6ad61d0
Final Commit	a695108

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

11	11	0	0
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	1
High	0
Medium	3
Low	3

Vulnerability	Severity	Status
F-2025-14231 - Lack of Access Control Enabling Unauthorized Credential Issuance and Revocation	Critical	Fixed
F-2025-14257 - Owner Authorization Allows Arbitrary Burning of Soulbound Tokens	Medium	Fixed
F-2025-14262 - Reusable Authentication Signatures Due to Missing Nonce	Medium	Fixed
F-2025-14269 - Possibility of Burning Incorrect Token Because of Mutable Credential Contract Address	Medium	Fixed
F-2025-14218 - Use of transfer() Instead of call() to Send Native Assets	Low	Fixed
F-2025-14259 - Inaccurate Credential Count Returned by View Functions	Low	Fixed
F-2025-14270 - Missing Active-State Check Allows Updates on Revoked DIDs	Low	Fixed
F-2025-14217 - Floating Pragma	Info	Fixed
F-2025-14254 - Custom Errors Can Be Used for Gas Efficiency	Info	Fixed
F-2025-14255 - Lack of Event Emitting for Key State Changes	Info	Fixed
F-2025-14261 - Approval Logic Conflicts With Soulbound Token Restrictions	Info	Fixed

Documentation quality

- Functional requirements are detailed.
 - Project overview is detailed
 - All roles in the system are described.
 - For each contract all futures are described
- Technical description is robust.
 - Run instructions are provided.
 - Technical specification is provided.
 - NatSpec is sufficient.

Code quality

- The development environment is configured.
- Code uses a modern Solidity version.
- Code does not follow best practices.

Test coverage

Code coverage of the project is **77.99%** (branch coverage).

- Deployment and basic user interactions are covered with tests.
- Negative cases coverage is missed.
- Interactions by several users are not tested thoroughly.

Table of Contents

System Overview	6
Privileged Roles	6
Potential Risks	7
Findings	8
Vulnerability Details	8
Disclaimers	33
Appendix 1. Definitions	34
Severities	34
Potential Risks	34
Appendix 2. Scope	35
Appendix 3. Additional Valuables	36

System Overview

The `SoulboundCredential` contract is a non-transferable ERC721 token system for credentials, where only authorized issuers or the owner can mint, update, or burn credentials. Credentials are permanently bound to users, cannot be transferred, and include metadata like type, data, issuer, and expiry. The contract tracks user stats, manages issuers, and allows pausing and emergency withdrawals.

The `DIDContract` is a decentralized identity registry that lets users create, update, and revoke DIDs, manage controllers, and issue or revoke credentials. It integrates with `SoulboundCredential` to issue non-transferable SBT credentials, supports authentication, tracks user stats, and allows the owner to set expiry limits, pause the contract, and withdraw funds.

Privileged roles

SoulboundCredential

- **Owner**
 - `addAuthorizedIssuer()` / `removeAuthorizedIssuer()` : Manage authorized issuers.
 - `setCredentialExpiryLimit()` : Set global expiry limit for credentials.
 - `updateUserReputation()` : Set user reputation score.
 - `pause()` / `unpause()` : Pause or unpause contract.
 - `emergencyWithdraw()` : Withdraw stuck ETH.
 - Can also mint, burn, and update any credential (full admin rights).
- **Authorized Issuer**
 - `mintCredential()` : Mint new credentials for users.
- **Credential Issuer (who issued a specific credential)**
 - `burnCredential()` : Burn credentials they issued.
 - `updateCredentialData()` : Update data for credentials they issued.
- **Credential Owner (user who owns a credential)**
 - `burnCredential()` : Burn their own credentials.

DIDContract

- **Owner**
 - `setSBTContract()` : Set the SBT contract address.
 - `setSBTEnabled()` : Enable/disable SBT integration.
 - `setCredentialExpiryLimit()` : Set global expiry limit for credentials.
 - `updateUserReputation()` : Set user reputation score.
 - `pause()` / `unpause()` : Pause or unpause contract.
 - `emergencyWithdraw()` : Withdraw stuck ETH.
- **DID Owner**
 - `addController()` / `removeController()` : Manage controllers for their DID.
- **DID Controller**
 - `updateDID()` : Update DID document.
 - `revokeDID()` : Revoke the DID

Potential Risks

- **Owner's Unrestricted State Modification:** The absence of restrictions on state variable modifications by the owner leads to arbitrary changes, affecting contract integrity and user trust, especially during critical operations like minting phases.
- **Absence of Time-lock Mechanisms for Critical Operations:** Without time-locks on critical operations, there is no buffer to review or revert potentially harmful actions, increasing the risk of rapid exploitation and irreversible changes.
- **Insufficient Multi-signature Controls for Critical Functions:** The lack of multi-signature requirements for key operations centralizes decision-making power, increasing vulnerability to single points of failure or malicious insider actions, potentially leading to unauthorized transactions or configuration changes.
- **Unrestricted DID Creation Leading to Counter Exhaustion:** The `createDID()` function relies on the `_didCounter` variable to assign identifiers to newly created DIDs but lacks access control, allowing any user to invoke it. A malicious actor could create an excessive number of DIDs, causing the counter to approach or reach the `uint256` limit. This would render the counter unusable and disrupt the system's ability to generate new DIDs.
- **Irrecoverable DID State After Revocation:** Once a DID is revoked, it cannot be reactivated, and the associated address is permanently blocked from creating a new DID. This creates an irreversible loss of identity for the user and may lead to unintended account lockouts or disruption of external systems relying on DID continuity.
- **Potential Credential ID Collision Due to Hash-Based Identifier Generation:**
Credential identifiers are derived from the output of `keccak256()`, which includes a string parameter as part of the input. Because the system relies solely on this hash value as the unique credential ID, there is a risk of collisions. In the event of a hash collision, a newly created credential may overwrite an existing one or become incorrectly associated with the same `credentialId`, leading to loss of data integrity and incorrect credential ownership.

Findings

Vulnerability Details

F-2025-14231 - Lack of Access Control Enabling Unauthorized Credential Issuance and Revocation - Critical

Description:

The `DIDContract` is intended to manage decentralized identity documents and to issue two types of credentials simple credentials and Soulbound Token (SBT) credentials. In the current implementation, the core functions that control this lifecycle (`issueCredential()`, `issueSBTCredential()`, and `revokeSBTCredential()`) are exposed without any meaningful access control. As a result, any arbitrary address can act as an issuer or revoker inside the system.

The `issueCredential()` function allows arbitrary callers to create unlimited credentials for any subject due to lack of an issuer check.

```
function issueCredential(  
    address subject,  
    string memory credentialType,  
    string memory credentialData,  
    uint256 expiresAt,  
    string[] memory proofTypes  
) external whenNotPaused nonReentrant {  
    ...  
}
```

A similar issue exists in `issueSBTCredential()`, although the underlying `SoulboundCredential` contract enforces an authorized-issuer rule on `mintCredential()` function, the wrapper function in `DIDContract` performs no access validation, effectively bypassing the expected authorization model.

```
function issueSBTCredential(  
    address subject,  
    string memory credentialType,  
    string memory credentialData,  
    uint256 expiresAt,  
    string[] memory proofTypes  
) external whenNotPaused sbtRequired nonReentrant {  
    ...  
  
    // Mint SBT through the SBT contract
```



```

sbtContract.mintCredential(
    subject,
    credentialType,
    credentialData,
    expiresAt,
    proofTypes
);
}

```

Revocation of the SBT credential has the same problem. The `revokeSBTCredential()` function can be triggered by any address, enabling unauthorized destruction of SBTs, loss of legitimate credentials, and manipulation of users' reputational history.

```

function revokeSBTCredential(uint256 tokenId)
    external
    whenNotPaused
    sbtRequired
{
    // Get the owner before burning
    address tokenOwner = sbtContract.ownerOf(tokenId);

    // Burn SBT through the SBT contract
    sbtContract.burnCredential(tokenId);

    emit SBTRevoked(
        msg.sender,
        tokenOwner,
        tokenId,
        block.timestamp
    );
}

```

Overall, without proper access control, the trust model of the credential system collapses. Fake or misleading credentials can be issued freely, SBTs can be revoked without authorization, and user credential histories can be manipulated. The system becomes unreliable for verification or reputation purposes. This restricts protocol functionality and damages user trust.

Assets:

- `contracts/DIDContract.sol` [<https://github.com/ryt-io/DID-Contract->]

Status:

Fixed

Classification

Impact Rate:	5/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Critical

Recommendations

Remediation:	Consider implementing strict access control on all credential issuance and revocation functions in the <code>DIDContract</code> . Define a dedicated issuer role or mapping and ensure that only authorized issuers can call <code>issueCredential()</code> , <code>issueSBTCredential()</code> , and <code>revokeSBTCredential()</code> .
Resolution:	Fixed in <code>a695108</code> , the <code>authorizedIssuers</code> mapping was implemented that defined authorized issuers that can be added or removed by the contract owner. The <code>onlyAuthorizedIssuer()</code> modifier was added to the mention functions to avoid unauthorized credentials issuance.

Evidences

PoC

Reproduce:	The <code>IssueCredentialIssue_PoC.test.ts</code> file was introduced to demonstrate the issue: 1. Attacker address issue simple credential via <code>issueCredential()</code> function. 2. Attacker mint SBT spam credential. 3. Authorized issuer (owner) issue legitimate SBT credential. 4. Attacker revoke legitimate SBT credential by burning the SBT token.
-------------------	--

PoC code:

```
it("Anauthorized fake credential issuance and revocation", async function () {
  const proofTypes = ["Ed25519Signature2020"];

  // Attacker tries to issue a fake credential
  await expect(
    didContract.connect(attacker).issueCredential(
      attacker.address,
      "University Degree",
      '{"degree": "PhD", "university": "Fake University"}',
    ),
  ).rejects.toThrow();
});
```

```

        0,
        proofTypes
    )
).to.emit(didContract, "CredentialIssued");

expect(await didContract.getTotalCredentials()).to.equal(1);

// Issue Spam SBT credentials to victim by not authorized issuer
expect(await sbtContract.authorizedIssuers(attacker.address)).to.equal(false);

await didContract.connect(attacker).issueSBTCredential(
    user1.address,
    "Spam Credential",
    "{}",
    0,
    proofTypes
);

// Try to issue a legitimate credential
await didContract.connect(owner).issueSBTCredential(
    user1.address,
    "Educational Credential",
    '{"degree': 'Bachelor of Science', 'field': 'Computer Science'}",
    0,
    proofTypes
);

expect((await sbtContract.getUserCredentials(user1.address)).length).to.equal(2);

// Revoke legitimate credentials by attacker
let victimCredentials = await sbtContract.getUserCredentials(user1.address);

await didContract.connect(attacker).revokeSBTCredential(victimCredentials[victimCredentials.length - 1]);

expect((await sbtContract.getUserCredentials(user1.address)).length).to.equal(1);
victimCredentials = await sbtContract.getUserCredentials(user1.address);
let credentialInfo = await sbtContract.getCredentialInfo(victimCredentials[0]);
expect(credentialInfo.credentialType).to.equal("Spam Credential");
});

```

Results:

```
DIDContract
  IssueCredentialIssue_PoC
    ✓ Anauthorized fake credential issuance and revocation

1 passing (330ms)
```

Files:

IssueCredentialIssue_PoC.test.ts

F-2025-14257 - Owner Authorization Allows Arbitrary Burning of Soulbound Tokens - Medium

Description:

The `SoulboundCredential` contract extends the ERC721 standard and disables any token transfer after minting to ensure that each credential remains bound to the original recipient. The contract includes a `burnCredential()` function for destroying existing credentials.

```
function burnCredential(uint256 tokenId)
    external
    validCredential(tokenId)
    whenNotPaused
    nonReentrant
{
    require(
        ownerOf(tokenId) == msg.sender ||
        credentialInfo[tokenId].issuer == msg.sender ||
        msg.sender == owner(),
        "SoulboundCredential: Not authorized to burn this credential"
    );
    // ...
}
```

The access control logic in this function grants the contract owner the ability to burn tokens belonging to any address. As a result, soulbound tokens can be destroyed at any time by the owner. In the event of owner key compromise, the likelihood of unauthorized token destruction increases.

Assets:

- `contracts/SoulboundCredential.sol` [<https://github.com/ryt-io/DID-Contract->]

Status:

Fixed

Classification

Impact Rate: 5/5

Likelihood Rate: 3/5

Exploitability: Dependent

Complexity: Simple

Severity:

Medium

Recommendations**Remediation:**

It is recommended to remove the contract owner's ability to burn users' soulbound tokens entirely to preserve the intended permanence of credentials. If owner-level burning is required for operational reasons, the burn functionality should be restricted so that only tokens meeting predefined conditions—such as tokens that have reached an explicit expiration state—can be burned. This ensures that destruction of user credentials cannot occur arbitrarily and remains aligned with the intended lifecycle of the soulbound token.

Resolution:

Fixed in [a695108](#), the owner of the contract was excluded from the authorized accounts who can burn the credentials. Only owner of the credential or the credential issuer can burn:

```
if (
    ownerOf(tokenId) != msg.sender &&
    credentialInfo[tokenId].issuer != msg.sender
) {
    revert NotAuthorizedBurn();
}
```

[F-2025-14262](#) - Reusable Authentication Signatures Due to Missing Nonce - Medium

Description:

The `DIDContract` implements an `authenticate` function that verifies user authentication through an ECDSA signature constructed from the DID, verification method, timestamp, and `msg.sender`. However, the function does not incorporate any nonce, replay counter, or signature-tracking mechanism. As a result, any valid signature remains reusable for the entire duration of the `proof.timestamp` window. Because the message hash is deterministic and contains no unique per-request value, the contract accepts identical signatures multiple times without detecting replay attempts.

```
function authenticate(
    string memory did,
    AuthenticationProof memory proof
) external whenNotPaused validDID(did) notExpired(proof.timestamp + 300) {
    ...

    // Verify the signature
    bytes32 messageHash = keccak256(
        abi.encodePacked(
            did,
            proof.verificationMethod,
            proof.timestamp,
            msg.sender
        )
    );
    bytes32 ethSignedMessageHash = ECDSA.toEthSignedMessageHash(messageHash);
    address signer = ECDSA.recover(ethSignedMessageHash, proof.signature);

    require(
        signer == didDocuments[did].owner ||
        didControllers[didDocuments[did].owner].contains(signer),
        "DIDContract: Invalid signature"
    );

    // Update user stats
    _updateUserStats(msg.sender);

    emit AuthenticationSuccessful(msg.sender, did, block.timestamp);
}
```

The absence of replay protection allows repeated reuse of the same authentication signature by either the original caller or an attacker who

captures it. This enables unauthorized replay of authentication attempts, inflates user activity through repeated updates to internal stats, and produces misleading or noisy audit trails. Although the attack does not directly grant elevated privileges, it degrades the reliability of authentication records and may enable secondary misuse such as spamming contract events or triggering unintended rate-dependent logic.

Assets:

- contracts/DIDContract.sol [<https://github.com/ryt-io/DID-Contract->]

Status:**Fixed****Classification****Impact Rate:** 3/5**Likelihood Rate:** 5/5**Exploitability:** Semi-Dependent**Complexity:** Simple**Severity:** **Medium****Recommendations****Remediation:**

Incorporate nonce in signature validation. To align with best practices and provide additional security, a nonce parameter should be included in the `authenticate` function and utilized in the signature validation process. Each successful authentication should update the nonce, ensuring that previously issued signatures become invalid.

Resolution:

Fixed in `a695108`, the authentication hash now includes the nonce which is incremented each time when the signature was used ensuring that previously issued signatures become invalid:

```
bytes32 messageHash = keccak256(
    abi.encodePacked(
        did,
        proof.verificationMethod,
        proof.timestamp,
        msg.sender,
        proof.nonce
    )
);

authNonces[msg.sender] = authNonces[msg.sender] + 1;
```


[F-2025-14269](#) - Possibility of Burning Incorrect Token Because of Mutable Credential Contract Address - Medium

Description:

The `DIDContract` and `SoulboundCredential` contracts are designed to operate together. The soulbound credentials are minted through the `DIDContract`, which requires the credential contract address to be set beforehand. The `revokeSBTCredential()` function burns an existing ERC721 soulbound token by referencing its token ID.

```
function revokeSBTCredential(uint256 tokenId) external whenNotPaused
    sbtRequired {
        // Get the owner before burning
        address tokenOwner = sbtContract.ownerOf(tokenId);

        // Burn SBT through the SBT contract
        sbtContract.burnCredential(tokenId);

        emit SBTRevoked(msg.sender, tokenOwner, tokenId, block.timestamp);
    }
```

The credential contract address can be modified by the contract owner at any time.

```
function setSBTContract(address _sbtContract) external onlyOwner {
    require(_sbtContract != address(0),
        "DIDContract: Invalid SBT contract address");
    sbtContract = SoulboundCredential(_sbtContract);
    sbtEnabled = true;

    emit SBTContractSet(_sbtContract);
}
```

If multiple instances of the `SoulboundCredential` contract exist, a modification of the stored credential contract address while a `revokeSBTCredential()` transaction is pending in the mempool may result in a burn operation executed against a different contract instance. This enables a scenario in which a token with the same ID from another deployed instance is burned instead of the intended one.

Assets:

- contracts/DIDContract.sol [<https://github.com/ryt-io/DID-Contract->]

Status:

Fixed

Classification

Impact Rate:	5/5
Likelihood Rate:	3/5
Exploitability:	Dependent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation: It is recommended to add the soulbound token contract address as an argument to the `revokeSBTCredential()` function and use this value instead of the address stored in the `sbtContract` variable. This adjustment enables correct revocation of soulbound credentials in environments where multiple `SoulboundCredential` contract instances may exist.

Resolution: Fixed in `a695108`, the new `sbtAddr` param was introduced in the `revokeSBTCredential()` function that is used as the target address of the `SoulboundCredential` contract:

```
function revokeSBTCredential(address sbtAddr, uint256 tokenId)
    external
    whenNotPaused
    sbtRequired
    onlyAuthorizedIssuer
{
    if (sbtAddr == address(0)) revert InvalidSBTAddress();
    SoulboundCredential target = SoulboundCredential(sbtAddr);
    ...
}
```

F-2025-14218 - Use of transfer() Instead of call() to Send Native Assets - Low

Description:

In the `SoulboundCredential` and `DIDContract` contracts the `emergencyWithdraw()` function uses built-in `transfer()` function for recovering stuck ETH to the owner wallet. However, if the beneficiary is a smart contract (e.g. multisig wallet), they might not be able to receive these ETH due to the gas limit associated with the transfer function in Solidity.

```
function emergencyWithdraw() external onlyOwner {  
    payable(owner()).transfer(address(this).balance);  
}
```

The `transfer()` function was commonly used in earlier versions of Solidity for its simplicity and automatic reentrancy protection. However, it was identified as potentially problematic due to its fixed gas limit of `2300`.

The usage of `transfer()` function can lead to unintended function call revert when the receiving contract's `receive()` or `fallback()` functions require more than `2300` Gas for processing.

Assets:

- `contracts/DIDContract.sol` [<https://github.com/ryt-io/DID-Contract->]
- `contracts/SoulboundCredential.sol` [<https://github.com/ryt-io/DID-Contract->]

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 4/5

Exploitability: Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation:

Consider using built-in `call()` function instead of `transfer()` to transfer native assets. This method does not impose a gas limit, it provides greater

flexibility and compatibility with contracts having more complex business logic upon receiving the native tokens. When working with then `call()` function ensure that its execution is successful by checking the returned boolean value.

Resolution:

Fixed in `a695108`, the `call()` function were used to send native assets:

```
function emergencyWithdraw() external onlyOwner {
    uint256 amount = address(this).balance;
    (bool ok, ) = payable(owner()).call{value: amount}("");
    if (!ok) revert WithdrawFailed();
    emit EmergencyWithdraw(owner(), amount);
}
```

[F-2025-14259](#) - Inaccurate Credential Count Returned by View

Functions - Low

Description:

Each newly minted soulbound token ID is assigned based on a counter that is incremented on every mint operation, resulting in sequentially increasing token IDs. According to the NatSpec of the `getTotalCredentials()` function, it is expected to return the total number of existing credentials.

```
/**
 * @dev Get total number of credentials
 * @return The total count
 */
function getTotalCredentials() external view returns (uint256) {
    return _tokenIdCounter.current();
}
```

However, when a credential has been burned, the function still returns a value that includes burned credentials, because the `burnCredential()` function does not modify the `_tokenIdCounter` in any way. As a result, the returned value represents the number of minted tokens rather than the number of currently active credentials.

A similar condition is present in the `DIDContract`, where the `getTotalDIDs()` and `getTotalCredentials()` functions return the total number of issued identifiers without distinguishing between active and revoked entries. These view functions therefore expose counters representing issuance volume rather than the number of active records.

```
/**
 * @dev Get total number of DIDs
 * @return The total count
 */
function getTotalDIDs() external view returns (uint256) {
    return _didCounter.current();
}

/**
 * @dev Get total number of credentials
 * @return The total count
 */
function getTotalCredentials() external view returns (uint256) {
    return _credentialCounter.current();
}
```

Assets:

- contracts/SoulboundCredential.sol [<https://github.com/ryt-io/DID-Contract->]

Status:

Fixed

Classification**Impact Rate:** 2/5**Likelihood Rate:** 3/5**Exploitability:** Independent**Complexity:** Simple**Severity:** Low**Recommendations**

Remediation: It is recommended to align the function name and NatSpec with the actual behavior, for example by renaming the function to `getTokenIdCounter()` or `getMintedCredentials()`. Alternatively, the logic may be updated to track active credentials separately if the intended purpose is to return only non-burned/active credentials.

Resolution: Fixed in `a695108`, the separate functions were introduced to return amount of active credentials and DIDs:

```
// SoulboundCredential.sol
function getActiveDIDs() external view returns (uint256) {
    return activeDIDCount;
}
function getActiveCredentials() external view returns (uint256) {
    return activeCredentialCount;
}

// DIDContract.sol
function getActiveDIDs() external view returns (uint256) {
    return activeDIDCount;
}
function getActiveCredentials() external view returns (uint256) {
    return activeCredentialCount;
}
```

F-2025-14270 - Missing Active-State Check Allows Updates on Revoked DIDs - Low

Description:

`DIDContract` allows users to create and manage DIDs. Once issued, a DID can be updated via `updateDID()`, have controllers added or removed through `addController()` and `removeController()`, and can be revoked by the DID owner using `revokeDID()`. All these functions rely on the `validDID()` modifier, which only verifies that the DID exists. However, it does not verify whether the DID has been revoked.

```
modifier validDID(string memory did) {
    require(
        bytes(did).length > 0,
        "DIDContract: DID cannot be empty"
    );
    require(
        didDocuments[did].owner != address(0),
        "DIDContract: DID does not exist"
    );
    _;
}
```

Because revoked DIDs cannot be reactivated, allowing further modifications to them has no logical or practical effect. These operations silently succeed, waste gas, and may create confusion for integrators or off-chain systems relying on DID state consistency.

Assets:

- `contracts/DIDContract.sol` [<https://github.com/ryt-io/DID-Contract->]

Status:

Fixed

Classification

Impact Rate:	2/5
Likelihood Rate:	2/5
Exploitability:	Semi-Dependent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation:

Extend the `validDID()` modifier (or introduce an additional modifier) to ensure the DID is active before permitting any updates or controller changes. This prevents unnecessary calls on revoked DIDs and maintains consistent DID lifecycle state.

Resolution:

Fixed in [a695108](#), the `activeDID()` modifier was introduced and invoked in the `mention` function to ensure that DID is active before making any new changes related to the DID:

```
modifier activeDID(string memory did) {  
    if (!didDocuments[did].active) revert DIDInactive();  
    _;  
}
```


F-2025-14217 - Floating Pragma - Info

Description:

In Solidity development, the pragma directive specifies the compiler version to be used, ensuring consistent compilation and reducing the risk of issues caused by version changes. However, using a floating pragma introduces uncertainty, as it allows contracts to be compiled with any version within a specified range. This can result in discrepancies between the compiler used in testing and the one used in deployment, increasing the likelihood of vulnerabilities or unexpected behavior due to changes in compiler versions.

The project currently uses floating pragma declarations `^0.8.24` in its Solidity contracts. This increases the risk of deploying with a compiler version different from the one tested, potentially reintroducing known bugs from older versions or causing unexpected behavior with newer versions. These inconsistencies could result in security vulnerabilities, system instability, or financial loss. Locking the pragma version to a specific, tested version is essential to prevent these risks and ensure consistent contract behavior.

Assets:

- contracts/DIDContract.sol [<https://github.com/ryt-io/DID-Contract->]
- contracts/SoulboundCredential.sol [<https://github.com/ryt-io/DID-Contract->]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

It is recommended to lock the pragma version to the specific version that was used during development and testing. This ensures that the contract will always be compiled with a known, stable compiler version, preventing unexpected changes in behavior due to compiler updates. For example,

instead of using `^0.8.24`, explicitly define the version with `pragma solidity 0.8.24;`.

Resolution:

Fixed in `a695108`, the pragma was locked to `0.8.24`.

F-2025-14254 - Custom Errors Can Be Used for Gas Efficiency - Info

Description:

Starting from Solidity version `0.8.4`, the language introduced a feature known as "custom errors". These custom errors provide a way to define more descriptive and semantically meaningful error conditions without relying on string messages. Prior to this version, the `require` statement with string error messages was commonly used to handle specific conditions or validations. However, each unique string used as a revert reason consumes gas, increasing transaction costs.

Within the `DIDContract` and `SoulboundCredential` contract codebases, there are several `require` and `revert` statements with significant revert messages. These revert messages, while useful for debugging, can be gas-inefficient due to the consumption of gas for each unique string used:

```
require(
    subject != address(0),
    "DIDContract: Invalid subject address"
);
require(
    bytes(credentialType).length > 0,
    "DIDContract: Credential type cannot be empty"
);
require(
    _expiryLimit > 0,
    "SoulboundCredential: Expiry limit must be greater than 0"
);
require(
    _expiryLimit <= 3650 days, // Max 10 years
    "SoulboundCredential: Expiry limit too high"
);
revert("SoulboundCredential: Tokens are soulbound and cannot be transferred")
;
...
```

Custom errors, on the other hand, are identified by their name and the types of their parameters only, and they do not have the overhead of string storage. This means that, when using custom errors instead of `require` statements with string messages, the gas consumption can be significantly reduced, leading to more gas-efficient contracts.

Assets:

- `contracts/DIDContract.sol` [<https://github.com/ryt-io/DID-Contract->]
- `contracts/SoulboundCredential.sol` [<https://github.com/ryt-io/DID-Contract->]

Status: Fixed

Classification

Impact Rate: 1/5
Likelihood Rate: 3/5
Exploitability: Independent
Complexity: Simple
Severity: Info

Recommendations

Remediation: It is recommended to use custom errors instead of revert strings to reduce gas costs. Custom errors can be defined using the `error` keyword and can include dynamic information, for example:

```
error InvalidDID(string did);  
error InvalidSignature();  
...  
revert InvalidDID(did)  
revert InvalidSignature();
```

Custom errors are more cost-effective because they reduce the amount of data that needs to be stored and processed on the blockchain when handling exceptions. While `require` statements generate error messages stored as strings, which consume additional gas, custom errors are more compact and can save a significant amount of gas, especially in complex contracts with numerous validation checks.

Resolution: Fixed in [a695108](#), the custom errors were introduced in both contracts.

F-2025-14255 - Lack of Event Emitting for Key State Changes - Info

Description:

Several administrative functions within the `DIDContract` and `SoulboundCredential` codebases perform meaningful state changes without emitting events, which hinders off-chain observability.

In both `DIDContract` and `SoulboundCredential` contracts, the owner can arbitrarily change any user's reputation score, but these changes are not logged on-chain via events:

```
function updateUserReputation(address user, uint256 score) external onlyOwner
{
    require(
        score <= MAX_REPUTATION_SCORE,
        "DIDContract: Reputation score too high"
    );
    userStats[user].reputationScore = score;
}

function updateUserReputation(address user, uint256 score) external onlyOwner
{
    require(
        score <= MAX_REPUTATION_SCORE,
        "SoulboundCredential: Reputation score too high"
    );
    userStats[user].reputationScore = score;
}
```

In `DIDContract`, the owner can enable or disable SBT credential issuance without emitting any event to signal the configuration change:

```
function setSBTEnabled(bool _enabled) external onlyOwner {
    sbtEnabled = _enabled;
}
```

The absence of an event makes it difficult to track admin-specific actions off-chain, which reduces transparency and complicates monitoring and auditing. It is recommended to emit events for this operation to ensure proper visibility, support off-chain indexing, and facilitate debugging, especially for critical administrative and user-facing fund transfers.

Assets:

- `contracts/DIDContract.sol` [<https://github.com/ryt-io/DID-Contract->]
- `contracts/SoulboundCredential.sol` [<https://github.com/ryt-io/DID-Contract->]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Consider emitting events within the mentioned affected functions to enhance transparency and ensure reliable off-chain tracking of key state changes. Emitting events not only improves monitoring of critical storage variable updates but also supports indexing services, facilitates auditing, and provides better visibility into administrative actions.

Resolution: Fixed in `a695108`, the `ReputationUpdated()` and `SBTEnabledUpdated()` events were introduced and emitted in the mentioned functions.

F-2025-14261 - Approval Logic Conflicts With Soulbound Token Restrictions - Info

Description:

The `SoulboundCredential` contract implements an ERC721 token with all transfer functionality disabled after minting, creating a non-transferable token. All public transfer functions, including `safeTransferFrom()`, are overridden with a revert statement to prevent token movement.

```
function transferFrom(
    address,
    address,
    uint256
) public pure override {
    revert("SoulboundCredential: Tokens are soulbound and cannot be transferred");
}

function safeTransferFrom(
    address,
    address,
    uint256
) public pure override {
    revert("SoulboundCredential: Tokens are soulbound and cannot be transferred");
}

function safeTransferFrom(
    address,
    address,
    uint256,
    bytes memory
) public pure override {
    revert("SoulboundCredential: Tokens are soulbound and cannot be transferred");
}
```

However, the `approve()` and `setApprovalForAll()` functions remain enabled, allowing approvals to be set while providing no practical utility because transfers cannot be executed. This results in inconsistent behavior and unnecessary gas expenditure when these functions are called.

Assets:

- `contracts/SoulboundCredential.sol` [<https://github.com/ryt-io/DID-Contract->]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: It is recommended to override the `approve()` and `setApprovalForAll()` functions to maintain consistent non-transferable token behavior and to prevent unnecessary gas usage related to ineffective approval operations.

Resolution: Fixed in `a695108`, the `approve()` and `setApprovalForAll()` were disabled to maintain consistent non-transferable token behavior:

```
function approve(address, uint256) public pure override {
    revert TokenSoulbound();
}

function setApprovalForAll(address, bool) public pure override {
    revert TokenSoulbound();
}
```


Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hknio/severity-formula](https://github.com/hacken/severity-formula)

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/ryt-io/DID-Contract-
Initial Commit	6ad61d06ccaea148335a4160039e95560e90f895
Final Commit	a69510881dbb22b16ebb10a31cd977bc2e5bcbcd
Whitepaper	N/A
Requirements	README.md
Technical Requirements	README.md

Asset	Type
contracts/DIDContract.sol [https://github.com/ryt-io/DID-Contract-]	Smart Contract
contracts/SoulboundCredential.sol [https://github.com/ryt-io/DID-Contract-]	Smart Contract

Appendix 3. Additional Valuables

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the [NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment](#), and the [Penetration Testing Execution Standard \(PTES\)](#). These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.